I Gave JARVIS Hands-Free Control Over Writing! (EP 17) ■■

by **Pulse of AI** | Official Architecture & Production Release

■■ CRITICAL SYSTEM DEPENDENCY NOTICE:
JARVIS is engineered as a modular AI assistant. Modules developed in earlier installments (such as hardware_control, software_control, screen_vision, skill_synthesizer, and llm_router) are compulsory dependencies. Viewers MUST watch all previous videos (Episodes 1 to 16) and extract the exact source code from their respective pinned comments, or JARVIS will fail runtime compilation and will not function.

1. Episode 17 Viral Short Production Blueprint

Timestamp	Visual Framing & On-Screen Action	Dialogue / Spoken Audio Delivery
0:00 - 0:03	Cinematic cut to glowing ROG keyboard & desktop terminal window.	**You:** *"Tony Stark ne kabhi kuch type karne ke liye keyboard ko touch nahi kiya... toh hum kyun karein? Welcome to Episode 17 of making real-life JARVIS."*
0:03 - 0:10	Target application (Notepad/Word) spawns; text streams rapidly via ghost-typing.	**You:** *"JARVIS, future technology par 100 words ka proposal likho."* **JARVIS:** *"Drafting proposal, Sir."*
0:10 - 0:14	Complete document formatted on display; on-screen CTA overlay pops up.	**You:** *"Code ke liye comments mein JARVIS likho!"* **Overlay Banner:** ■ COMMENT "JARVIS" FOR CODE! ■■

2. Complete Series Architecture Catalog (Episodes 1 – 16)

Part	Direct YouTube Link	Part	Direct YouTube Link
Part 1	youtube.com/shorts/4fvNmeZ0E68	**Part 9**	youtube.com/shorts/9ATHesGlVdA
Part 2	youtube.com/shorts/s2sz62HNmbE	**Part 10**	youtube.com/shorts/g8v89plnk-Y
Part 3	youtube.com/shorts/qhaxKI-aocU	**Part 11**	youtube.com/shorts/FJvn6oonfk8
Part 4	youtube.com/shorts/WvV8ss0jn2o	**Part 12**	youtube.com/shorts/nbP_oir2YKc
Part 5	youtube.com/shorts/7DCTFvycGe4	**Part 13**	youtube.com/shorts/8vIIph3guxk
Part 6	youtube.com/shorts/ek-iZBw-vjY	**Part 14**	youtube.com/shorts/nZkJVA_NCbc
Part 7	youtube.com/shorts/6kbS4zYSN2Q	**Part 15**	youtube.com/shorts/O9v1zEAa0X0
Part 8	youtube.com/shorts/FWWldND5qO4	**Part 16**	youtube.com/shorts/BkD897ekPeY

3. Complete Code: app_automator.py (Autonomous App Dispatcher & Ghost Typing)

```
"""
app_automator.py
Autonomous Application Dispatcher & High-Tech Typing Engine for JARVIS (Episode 17).
Features asynchronous background execution for immediate voice replies,
strict foreground window verification, and rapid clipboard-burst typing.
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import time
import threading
import subprocess
import pyautogui
import pyperclip
import pygetwindow as gw

APP_TARGETS = {
    "notepad": {"cmd": "notepad.exe", "title": "Notepad"},
    "word": {"cmd": "start winword", "title": "Word"},
    "powerpoint": {"cmd": "start powerpnt", "title": "PowerPoint"},
    "calc": {"cmd": "calc.exe", "title": "Calculator"}
}

def generate_stunning_document(topic: str, word_count: int = 150, mode: str = "essay") -> str:
    """Creates a beautifully structured, multi-section document matching the target word count."""
    clean_topic = topic.strip().title() if topic.strip() else "Autonomous Intelligence"
    timestamp = time.strftime("%Y-%m-%d | %H:%M:%S")

    header = (
        "================================================================================\n"
        f"      ■ JARVIS INTELLIGENCE BRIEF // CORE PROTOCOL: {clean_topic.upper()}\n"
        f"      TIMESTAMP: {timestamp} // CLEARANCE: LEVEL 5 // STATUS: VERIFIED\n"
        "================================================================================\n\n"
    )

    abstract = (

        "[01] EXECUTIVE ABSTRACT\n"
        "--------------------------------------------------------------------------------\n"
        f"The acceleration of {clean_topic} represents a fundamental transition in modern\n"
        "computational workflows. By establishing closed-loop sensory feedback and eliminating\n"
        "deterministic friction, autonomous runtime systems transition from passive data\n"
        "listeners into proactive system operators.\n\n"
    )

    pillars = (
        "[02] ARCHITECTURAL PILLARS & CORE METRICS\n"
        "--------------------------------------------------------------------------------\n"
        f"• Neural Orchestration : Direct algorithmic grounding mapped to {clean_topic}.\n"
        "• Latency Optimization : Sub-1.5s bidirectional response loops across active nodes.\n"
        "• Desktop Telemetry    : Native OS window binding and memory-safe code injection.\n"
        "• Operational Fault-Tol: Multi-tier cloud failover ensuring continuous uptime.\n\n"
    )

    deep_analysis_base = (
        f"Deploying {clean_topic} within real-time desktop environments bridges the\n"
        "traditional disconnect between conversational intent and physical digital execution.\n"
        "Through low-overhead application hooking, the architecture monitors system state,\n"
        "parses complex input vectors, and drafts high-density technical deliverables without\n"
        "requiring manual human intervention. This continuous feedback loop ensures that every\n"
        "operation maintains absolute precision under fluctuating hardware constraints.\n\n"
    )

    trajectory = (
        "[03] STRATEGIC FORWARD TRAJECTORY\n"
        "--------------------------------------------------------------------------------\n"
        f"As systems aligned with {clean_topic} mature, manual interface management will\n"
        "become obsolete. Autonomous workflows will seamlessly bridge voice commands,\n"
        "visual reasoning, and rapid mechanical typing to establish a truly frictionless\n"
        "digital workspace.\n\n"
        "--------------------------------------------------------------------------------\n"
        "// END OF TRANSMISSION // COMPILED AUTONOMOUSLY BY JARVIS // PULSE OF AI\n"
```

```
        "================================================================================\n"
    )

    current_text = header + abstract + pillars + trajectory
    current_words = len(current_text.split())

    body_expanded = ""
    while (current_words + len(body_expanded.split())) < word_count:
        body_expanded += deep_analysis_base
        if (current_words + len(body_expanded.split())) >= word_count:
            break

    if body_expanded:
        full_document = (
            header
            + abstract
            + pillars
            + "[03] DETAILED SYSTEM ANALYSIS\n--------------------------------------------------------------------------------\n"
            + body_expanded
            + trajectory
        )
    else:
        full_document = header + abstract + pillars + trajectory

    return full_document

def ensure_target_focused(target_title: str) -> bool:
    """Verifies and forces the target window into the foreground."""
    windows = [w for w in gw.getAllWindows() if target_title.lower() in w.title.lower()]
    if not windows:
        return False
    target_win = windows[0]
    try:
        if target_win.isMinimized:
            target_win.restore()

        target_win.activate()
        time.sleep(0.15)
        return True
    except Exception:
        return False

def fast_ghost_type(text: str, target_title: str, word_delay: float = 0.012, line_delay: float = 0.04):
    """Types into the target application with focus safety locks."""
    lines = text.split("\n")
    for line in lines:
        if not ensure_target_focused(target_title):
            print(f"[AUTOMATOR] Target window '{target_title}' lost focus. Halting injection to protect active window.")
            break

        if not line.strip():
            pyautogui.press("enter")
            continue

        if any(border_char in line for border_char in ["===", "---"]):
            pyperclip.copy(line)
            pyautogui.hotkey("ctrl", "v")
            pyautogui.press("enter")
            time.sleep(0.03)
            continue

        words = line.split(" ")
        for i, word in enumerate(words):
            pyperclip.copy(word + (" " if i < len(words) - 1 else ""))
            pyautogui.hotkey("ctrl", "v")
            time.sleep(word_delay)

        pyautogui.press("enter")
        time.sleep(line_delay)

def _compose_worker(target_key: str, topic: str, word_count: int, mode: str):
```

```
    """Background worker handling application spawning, focus, and stream typing."""
    app_meta = APP_TARGETS[target_key]

    try:
        subprocess.Popen(app_meta["cmd"], shell=True)
        time.sleep(1.2)
    except Exception:
        target_key = "notepad"
        app_meta = APP_TARGETS["notepad"]
        subprocess.Popen(app_meta["cmd"], shell=True)
        time.sleep(1.2)

    if not ensure_target_focused(app_meta["title"]):
        print(f"[AUTOMATOR] Unable to acquire focus on {app_meta['title']}.")
        return

    document_content = generate_stunning_document(topic, word_count=int(word_count), mode=mode)
    fast_ghost_type(document_content, target_title=app_meta["title"], word_delay=0.012, line_delay=0.03)

def open_and_compose(app_name: str, topic: str, word_count: int = 150, mode: str = "essay") -> str:
    """Dispatches the document composition task to a background worker and returns immediately."""
    raw_app = app_name.strip().lower() if app_name else "notepad"

    target_key = "notepad"
    for key in APP_TARGETS.keys():
        if key in raw_app:
            target_key = key
            break

    app_meta = APP_TARGETS[target_key]

    worker = threading.Thread(
        target=_compose_worker,
        args=(target_key, topic, word_count, mode),
        daemon=True

    )
    worker.start()

    return f"Dispatching background automation: Opening {app_meta['title']} and composing {word_count} words on '{topic}' right
now, sir."
```

4. Complete Code: agent.py (Gemini Live Engine with Cyberpunk HUD & Tool Dispatcher)

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode 17)
Autonomous App Orchestration & Live Ghost Typing + Dual-Terminal Input (PowerShell + HUD)
Fast Cloud Semantic Router (Gemini 3.6 Flash / Claude) & Self-Modifying Skill Synthesizer
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import math
import struct
import threading
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,

    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)
```

```
# Episode 17: Autonomous App Composer & High-Speed Typing Engine
from app_automator import (
    open_and_compose,
)

load_dotenv()

console = Console()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
media_is_playing_state = False

# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        # Cyberpunk floating overlay styling
        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)

        # Header Bar Frame (Houses Title & Compact Switch)
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[—] COMPACT",
            font=("Consolas", 8, "bold"),
            bg="#1E293B",
            fg="#38BDF8",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=6,
            command=self.toggle_mode
        )
        self.toggle_btn.pack(side=tk.RIGHT)

        self.sub_header = tk.Label(
            self.root,
            text="Dual-Mode Voice & Keyboard Station • Pulse of AI",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
```

```
        )
        self.sub_header.pack(pady=(0, 4))

        # Translucent Chat Log Display
        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        # Dialogue & status tags
        self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("user_text", foreground="#A78BFA", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

        # Keyboard Command Input Bar (Always accessible even in mini-mode)
        self.input_frame = tk.Frame(self.root, bg="#0B0F19")
        self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            self.input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",

            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
        self.entry_box.bind("<Return>", self.on_send)

        self.send_btn = tk.Button(
            self.input_frame,
            text="TRANSMIT",
            font=("Consolas", 9, "bold"),
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        """Switches between Full Log HUD and Sleek Floating Input Bar."""
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True, before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False
```

```
    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)


def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)

def calculate_rms(data: bytes) -> float:
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150, doc_type: str = "essay") -> str:
    """Opens Notepad, Word, or PowerPoint and writes an essay or notes matching the exact target word count."""
    return open_and_compose(app_name, topic, word_count=int(word_count), mode=doc_type)

# Master Tools Manifest
tools = [
    open_website,
```

```
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background media state monitor

async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk: Hold [Right Ctrl] or type in Termin
al/HUD.[/bold yellow]")
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or type below.", "system")
                else:
                    console.print("\n[bold green]■ [MEDIA IDLE] Hands-Free Mode Active. Speak or type freely.[/bold green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
        except Exception:
            pass
        await asyncio.sleep(1.0)

# Asynchronous Terminal Input Listener
async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

# Routes typed inputs from both Terminal and HUD straight into the Live WebSocket session
async def handle_unified_text_input(session, text_queue):
```

```
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]■■ [TRANSMITTED INPUT][/bold magenta] {text}")
                await session.send_realtime_input(text=text)
        except Exception:
            await asyncio.sleep(0.05)

# Microphone streaming loop
async def send_audio(session):
    global model_is_speaking, media_is_playing_state
    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )
    console.print("[cyan]■ [SYSTEM ONLINE] Dynamic Voice & Terminal Input Active...[/cyan]")

    silence_frame = b'\x00' * (CHUNK * 2)

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )

            # Drop mic packets while JARVIS answers
            if model_is_speaking:
                await session.send_realtime_input(
                    audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                )
                continue


            # Push-to-Talk during media playback
            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )
            # Dynamic Hands-Free threshold
            else:
                rms = calculate_rms(data)
                if rms > 450:
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )

    except asyncio.CancelledError:
        raise
    finally:
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

# Inbound stream receiver: Handles tools, synchronized typewriter streaming, and audio
async def receive_audio(session, audio_queue):
    global model_is_speaking
```

```
    try:
        while True:
            async for response in session.receive():
                # 1. Tool Calls Execution
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":

                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts, direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view, mode)
                        elif call.name == "set_volume":
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
                            level = call.args.get("level", 50)
                            result = await asyncio.to_thread(set_brightness, level)
                        elif call.name == "get_brightness":
                            result = await asyncio.to_thread(get_brightness)
                        elif call.name == "get_battery_status":
                            result = await asyncio.to_thread(get_battery_status)
                        elif call.name == "get_system_vitals":
                            result = await asyncio.to_thread(get_system_vitals)
                        elif call.name == "get_system_temperatures":
                            result = await asyncio.to_thread(get_system_temperatures)
                        elif call.name == "analyze_screen":
                            prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                            result = await analyze_screen(prompt)
                        elif call.name == "find_ui_element":
                            element_name = call.args.get("element_name", "")
                            result = await find_ui_element(element_name)
```

```
                        elif call.name == "skip_youtube_ad":
                            result = await skip_youtube_ad()
                        elif call.name == "evolve_skill":
                            skill_description = call.args.get("skill_description", "")
                            result = await asyncio.to_thread(evolve_skill, skill_description)
                        elif call.name == "list_active_skills":
                            result = await asyncio.to_thread(list_active_skills)
                        elif call.name == "query_deepseek_reasoning":
                            query = call.args.get("query", "")
                            result = await asyncio.to_thread(query_deepseek_reasoning, query)
                        elif call.name == "create_document_and_write":
                            target_app = call.args.get("app_name", "notepad")
                            target_topic = call.args.get("topic", "")
                            target_words = call.args.get("word_count", 150)
                            target_mode = call.args.get("doc_type", "essay")
                            result = await asyncio.to_thread(create_document_and_write, target_app, target_topic, target_words,
target_mode)

                        console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/green]")
                        log_to_gui(f"■■ [{call.name}] {result}", "tool")

                        function_responses.append(
                            types.FunctionResponse(
                                name=call.name,
                                id=call.id,
                                response={"result": result},
                            )
                        )

                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                # 2. Content Streaming & Real-Time Sync Transcriptions
                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:

                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False

                    # User Voice Transcription
                    if server_content.input_transcription:
                        user_spoken = server_content.input_transcription.text
                        if user_spoken and user_spoken.strip():
                            console.print(f"\r[bold white][USER (Voice)][/bold white] {user_spoken.strip()}")
                            log_to_gui(f"YOU (Voice): {user_spoken.strip()}", "user_voice")

                    # Live Model Stream Chunk Handlers (Typewriter Animation)
                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                            stream_token_to_gui("JARVIS: ", "jarvis_header")

                        for part in model_turn.parts:
                            if part.text:
                                sys.stdout.write(part.text)
                                sys.stdout.flush()
                                stream_token_to_gui(part.text, "jarvis_stream")

                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)

                    # Fallback output audio transcription tokens
                    if hasattr(server_content, "output_transcription") and server_content.output_transcription:
                        transcribed_token = server_content.output_transcription.text
                        if transcribed_token:
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
```

```
                                stream_token_to_gui("JARVIS: ", "jarvis_header")

                            sys.stdout.write(transcribed_token)
                            sys.stdout.flush()
                            stream_token_to_gui(transcribed_token, "jarvis_stream")

                    # Turn complete: flush line break to prepare next interaction
                    if server_content.turn_complete:
                        model_is_speaking = False
                        stream_token_to_gui("\n\n", "system")
                        console.print("\n[cyan]■ [LISTENING / READY FOR INPUT] Awaiting instructions...[/cyan]")

    except asyncio.CancelledError:
        raise
    finally:
        await audio_queue.put(None)

# Audio output player
async def play_audio(audio_queue):
    speaker_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_OUT,
        output=True,
        frames_per_buffer=CHUNK,
    )
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:

        try:
            speaker_stream.stop_stream()
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting, text_queue):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()
        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session))
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task, return_exceptions=T
rue)
```

```
async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Persona: Calm, highly sophisticated, articulate, and respectful, exactly like Tony Stark's JARVIS. "
                        "Always address the user as 'sir'. "
                        "CRITICAL LANGUAGE ADAPTATION RULE: "
                        "Match the exact language and dialect spoken or typed by the user. "
                        "If the user speaks or types in Hindi or Hinglish (e.g., 'Notepad kholo aur essay likho', 'kya hal hai')
, "
                        "you MUST reply completely in fluent, natural Hindi or polished Hinglish with 'sir' (e.g., 'Ji sir, Note
pad kholkar essay likh raha hoon'). "
```

```
                        "Never force English if the user initiated the conversation in Hindi. "
                        "If the user speaks in English, reply in an articulate, polite Indian English accent. "
                        "Omni-Input Rule: The user can interact with you via voice microphone, typed text in the PowerShell term
inal, "
                        "or typed text in the Cyberpunk HUD console. Treat all three input sources with identical high priority.
 "
                        "Always respond aloud via audio so the user hears your voice through their speakers, "
                        "while keeping your responses concise so they stream cleanly as text on the HUD and terminal. "
                        "Never output duplicate answers, redundant restatements, or multiple choices. State the final answer onl
y once. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site name]' (or in Hi
ndi: 'Ji sir, [site name] khol raha hoon'). "
                        "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir, opening [app na
me]' (or in Hindi: 'Ji sir, [app name] start kar raha hoon'). "
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game, call close_app
lication with the app name. "
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan threads, or opt
imize RAM, call purge_background_tasks. "
                        "5. When asked what song, video, or track is playing, or what media is active, call get_current_media_in
fo. "
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or mute/unmute audio,
call control_media with the action ('play_pause', 'next', 'previous', 'stop', 'mute'). "
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with direction='ne
xt'. "
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call skip_video wi
th direction='next'. "
                        "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with action='for
ward'. "
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call toggle_video_view. "
                        "11. When setting volume, call set_volume. "
                        "12. When asked what the volume level is, call get_volume. "
                        "13. When setting brightness, call set_brightness. "
                        "14. When asked what the brightness is, call get_brightness. "
                        "15. When asked about battery, power, or charging state, call get_battery_status. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running, call get_system
_temperatures. "
                        "17. When asked for full diagnostics, performance vitals, system status, or complete CPU/GPU/RAM stats,
call get_system_vitals. "
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what is visible o
n display, call analyze_screen. "
                        "19. When asked to locate or find a general UI icon, link, or window element, call find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip video ad', 'cli
ck skip', or any variation of skipping/bypassing advertisements, you MUST IMMEDIATELY call skip_youtube_ad. "
                        "21. When asked to learn a new skill, teach yourself a tool, write new Python code into your core, or ev
olve a capability, call evolve_skill. "
                        "22. When asked what dynamic skills, new tools, or custom functions you have learned, call list_active_s
kills. "
                        "23. When asked to solve complex math, reason deeply, calculate probabilities, or when the user says 'as
k DeepSeek' or 'use R1', call query_deepseek_reasoning and state only the resulting answer directly to the user once, sir. "
                        "24. When the user asks to open Notepad, Word, or PowerPoint and write an essay or notes, but has NOT sp
ecified the length, DO NOT call create_document_and_write immediately. Ask aloud: 'How many words would you like the essay to be
, sir?' (or in Hindi: 'Aap kitne shabdon ka essay chahte hain, sir?'). Once the user specifies the count, call create_document_a
nd_write with the app_name, topic, word_count, and doc_type. "
                        "General Rule: If the user asks a conversational question like 'can you hear me' or 'kya meri awaz aa ra
hi hai', simply respond politely in that same language. "
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
```

```
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]■■■
```